

Project: Document Q&A bot

LESSON 15 · MODULE 3 — KNOWLEDGE & MEMORY

What you'll learn

- How to assemble a production-shaped Q&A bot from everything in videos 10–14 — **no new APIs, just orchestration**
- The **"ground every turn"** pattern: retrieval grounds each response in facts while memory threads intent across turns
- How a single chat function combines a knowledge base, per-call retrieval config, conversation memory, and strict refusal instructions
- Where to take it next (FastAPI endpoint, multi-KB routing, multi-tenant front door)

Key concepts

This capstone is orchestration, not new surface area. A user sends a message; the agent retrieves from the knowledge base with per-call config (top-k, MMR, score threshold); memory threads the `session_id` so the agent knows what the user said earlier; the agent answers grounded in retrieved facts *plus* the conversation so far. Every piece is something you already built in this block.

Two pieces of state, independent but cooperating. The knowledge base is *static* — same docs, same retrieval API, call after call. Conversation memory is *dynamic* — it grows every turn, scoped by `session_id`. Retrieval grounds each individual response in facts; memory keeps the user's intent threaded across responses. Call the pattern **"ground every turn."**

Strict instructions are the safety floor. The agent answers only from VeriStack documentation in context and falls back to a fixed refusal phrase when retrieval comes up empty — enterprise customers can't have a support bot that confabulates. The capstone softens it slightly to permit *synthesis*: "how do I X" gets a focused answer from the relevant doc; "what's different about Y" gets a comparison across docs. That split stops the agent from dumping every retrieved doc on a specific question while still letting it compare when asked.

Per-call retrieval config is the lever. Same knowledge base, different `with_config()` per agent or per turn — pull five chunks here, tune MMR there. Production code tunes this per use case (power users vs. casual users) without ever mutating the KB.

This shape scales to a real product. One knowledge base serves a thousand users, each with their own session. Wrap the chat function in a FastAPI endpoint, add a routing layer in front for multi-tenant

TAKEAWAY

This capstone is orchestration, not new surface area.

TAKEAWAY

Two pieces of state, independent but cooperating.

TAKEAWAY

Strict instructions are the safety floor.

TAKEAWAY

Per-call retrieval config is the lever.

TAKEAWAY

This shape scales to a real product.

isolation, swap strict refusal for a softer fallback when the user is internal, or add multiple KBs per product area and pick which to query — the architecture holds.

Code shown in this lesson

PYTHON

```
# Same five VeriStack docs from video 12 (SSO, rate limits, data export, audit logs, RBAC).
kb = studio.create_knowledge_base(name="veristack-docs") # defaults to Qdrant
for doc in veristack_docs:
    kb.add_text(doc)

# Agent: strict-but-synthesizing instructions + a rolling memory window.
agent = studio.create_agent(
    name="veristack-qa", provider="gpt-4o",
    role="VeriStack support agent",
    goal="answer accurately from VeriStack docs; refuse cleanly when unsupported",
    instructions=(
        "Answer only from the VeriStack documentation in context. "
        "If the answer isn't there, reply: "
        "'I don't have that information in the documentation I can see.' "
        "For 'how do I...' give a focused answer from the relevant doc; "
        "for 'what's different about...' synthesize across docs."
    ),
    memory=20,
)
```

PYTHON

```
# One chat function = all four building blocks doing their job.
def ask(message, session_id):
    tuned = kb.with_config(top_k=5, retrieval_type="mmr", score_threshold=0.3) # video 12
    resp = agent.run(message, knowledge_bases=[tuned], session_id=session_id) # video 14 threads
    print(resp.response)
    return resp
```

PYTHON

```
# Four turns, each building on the last. Turn 3 synthesizes across docs + remembers turn 1.
sid = "acme-eval"
ask("We're evaluating the Business plan. How does SSO work?", sid) # focused, from SSO doc
ask("What are the rate limits?", sid) # focused, from rate-limit doc
ask("What's actually different between plans for us?", sid) # synthesis across 3 docs
ask("Summarize what I should tell my team.", sid) # memory threads the whole ev
```

TRY THIS

- 1 Build the bot with all five docs and run the four-turn script. Confirm turn 3 *compares* across docs while the "how do I" turns stay focused on one doc — that's the synthesis split working.
- 2 Ask something the corpus doesn't cover and verify the agent hits the refusal phrase instead of inventing an answer. Then loosen the instructions to a softer "internal user" fallback and compare.

3

Wrap `ask()` in a FastAPI `POST /chat` endpoint taking `{message, session_id}`. Hit it from two different `session_id` s and confirm clean per-user isolation — you're at MVP.